Efficient Global Back-Projection on an FPGA

Dan Pritsker
Altera Corporation
San Diego, CA USA
dpritske@altera.com

***Abstract*— The Global Back-Projection (GBP) Algorithm has been known for use in Synthetic Aperture Radar (SAR) image formation for many years. But, despite many advantages compared to other algorithms, GBP usage is limited. It requires significant computational complexity and consumes more power. This paper presents a high-performance architecture which allows efficient and scalable implementation of the GBP algorithm on a Field Programmable Gate Array (FPGA). Such an architecture is based on a systolic array that performs back-projection in a pipelined way, using parallelism over image pixels and radar pulses. A GBP systolic array is built using specially designed back-projection streaming cores. The cores utilize a floating-point calculation on the FPGA to achieve accurate and high-dynamic range in GBP integration. Results presented herein, show the achieved performance of GBP-based image formation using an FPGA and details the power consumption of the described implementation. Comparison is made to other computational platforms in order to determine advantage for real-time and power constrained SAR applications and platforms.**

Keywords—Synthetic Aperture Radar, SAR, Back-Projection Algorithm, Image Formation, FPGA, GPU, Floating-Point;

I. INTRODUCTION

Synthetic Aperture Radar (SAR) is used in a wide variety of applications to perform surveillance with the radar. Along with military surveillance SAR has multiple civil applications in medical tomography, agriculture and geology [5]. The process of creating visual images out of radar samples is called image formation. Many image formation algorithms are used in SAR systems. Each one of them has certain advantages and shortcomings, making them more suitable for varied applications. The Global Back-Projection (GBP) algorithm is a time-domain based algorithm. The GBP algorithm does not make any assumptions or give any approximations pertaining to the image or collection model. Thus it is considered the "golden reference" for SAR image quality. It was researched along the years to show the formatted image superiority in comparison to other algorithms, along with other operational benefits [3, 9]. It doesn't limit the collection to strip-map or spotlight, allowing collection flexibility. Another advantage is the fact that it can overlay the formatted image over topographic maps. But, the usage of GBP is limited because of the computational complexity associated with it. The number of operations required to format images using GBP is scaled with $O(N^3)$. The availability of General Purpose Graphical Processing Units (GPGPUs) has triggered research in the area of GBP implementation [1, 6, 7]. GBP algorithm can be partitioned for parallel implementation purposes, and mapped to GPGPU cores. Despite the wide research, the GBP algorithm didn't find its way into many platforms, especially those requiring Real-Time and Size, Weight and Power (SWaP) constrained applications. FPGAs are known to offer a high-performance and power efficient processing architecture. The GBP algorithm implementation was mainly researched in [2, 5, 8] to show its advantages paired with FPGAs for image formation. Although demonstrating performance benefits, this research didn't cover the potential advantages related to SWaP and power consumption, in particular.

GBP algorithm consists of multiple steps. It has few nonlinear functions with wide dynamic range. The GBP is built on the concept of coherent integration that is done in a recursive way, which makes it very suitable to perform calculations using floating-point. With the availability of FPGA components with integrated floating-point capabilities built into device silicon, FPGAs become even more optimal for use with the GBP algorithm. In contrast to previous approaches that implemented floating-point "emulated" operations, implemented in general purpose logic resources, it is now possible to collapse a significant amount of processing into specially-designed compact silicon blocks. One can gain a power savings, and also integrate more GBP processing into the same processing node, which improves other aspects of SWaP, as well.

II. BACKGROUND

A. Global Back-Projection Algorithm

GBP is a time-domain algorithm that is close in its make-up to the physical model of the radar waves that are emitted, reflected, and received by radar antenna. The radar emits the electromagnetic pulse and receives echoes of reflected waveforms as showed in Fig. 1.

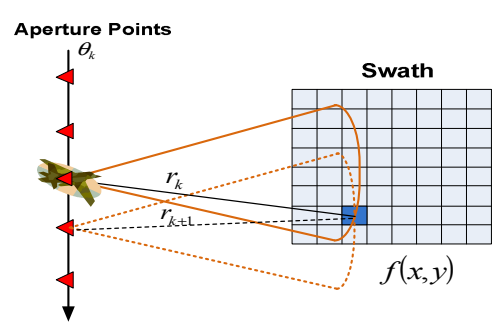

Fig. 1. Back-Projection imaging model

Due to the nature of wave's propagation, the reflected wave received by radar, corresponds to reflectors along the spherical surface with a given radius r from a transmitter. This implies poor cross-range resolution. To achieve fine cross-range resolution, the imaging is done from multiple points along the track called "Aperture points". The projection of radar echoes is done for each one of the image pixels. If a swath has a reflector in a corresponding pixel, it will create constructive interference from multiple aperture points, and appear as an active pixel in a formatted image. If this were not the case, the interference would be destructive and the active pixel wouldn't appear. The GBP final image is given by (1).

$$f(x,y)=\sum_{k} g_{x,y}(r_k,\theta_k)\cdot e^{j\omega\cdot 2\cdot|\vec{r}_k|} \qquad (1)$$

$f(x,y)$ - Formatted image, an output from algorithm

θ_k - Aperture point.

r_k - Range from pixel (x,y) to aperture point θ_k.

ω - Minimal angular velocity of radar waveform.

$g_{x,y}(r_k,\theta_k)$ - Reflection received by radar at r_k at θ_k.

From (1) it can be seen that the formation is created for each one of the pixels without any dependency on other pixels, which presents a property for potential parallelization of the algorithm. For each one of the pixels of the image, the integration is done along k aperture points. This is another dimension, where the algorithm can be parallelized. It is possible to calculate the integrants in parallel manner, and then to perform the integration step.

The reflections from the swath are received in the radar receive path and represented by $g_{x,y}(r_k,\theta_k)$. In radar systems, the received signal is a discrete signal passing multiple front-end processing stages that condition the signal and perform pulse compression. The resulting signal is represented as a discrete series of samples referred as a "Range Bins". Each sample corresponds to a received reflected signal at a discrete distance point from the radar. The range bins discrete points don't necessarily aligned with a range r_k, the distance from aperture point θ_k to every pixel in the image [3]. Thus, interpolation is required to obtain a more accurate estimation on the reflected signal that corresponds to r_k. Typically, linear interpolation based on immediate neighbor samples is sufficient for GBP.

B. Gotcha Volumetric SAR Data Set

This dataset is a publicly released collection of SAR phase history data collected at X-band. This set is owned by the Air Force Research Laboratory (AFRL). It contains imaging scenes with multiple civilian vehicles and calibration targets and covers the full azimuth extent. In this research, this dataset was used to develop and verify the implementation described below [10].

III. Implementation Scope

The following sections would describe the details of the proposed implementation of GBP on FPGA.

A. Architecture of GBP Engine

GBP is a memory-intensive algorithm. For each pixel, it has to access a significant number of radar samples from each one of the aperture points. A typical number can be many thousands of aperture points. The formatted image size is very significant which forces practical systems to use SDRAM type memory. Those memories have long latency which may limit performance of the system, when the algorithm is sensitive to latency. Memory accesses also have power consumption implications. The memory management and topologies for GBP were discussed in [6, 7].

In order to make GBP implementation efficient, the memory access bandwidth and pattern have to be balanced with the processing capabilities of the processing platform. A FPGA architecture fits well with such requirements due to the architecture's ability to store the samples as they are passing across the deep processing pipeline. Such capability minimizes access bandwidth to memory. In addition, it allows a sequential access pattern to the pixel memory which fits the nominal access pattern to SDRAM.

The block diagram of the GBP engine is showed in Fig. 2. The pixels are streamed across a specially-designed GBP engine. The engine is built as a systolic array of dedicated cores which perform back-projection. At every clock cycle, the pixels are progressing through the array, while new pixels are pushed into the engine. The data independence described above allows executing the pipeline without any stalls, which makes the engine very efficient.

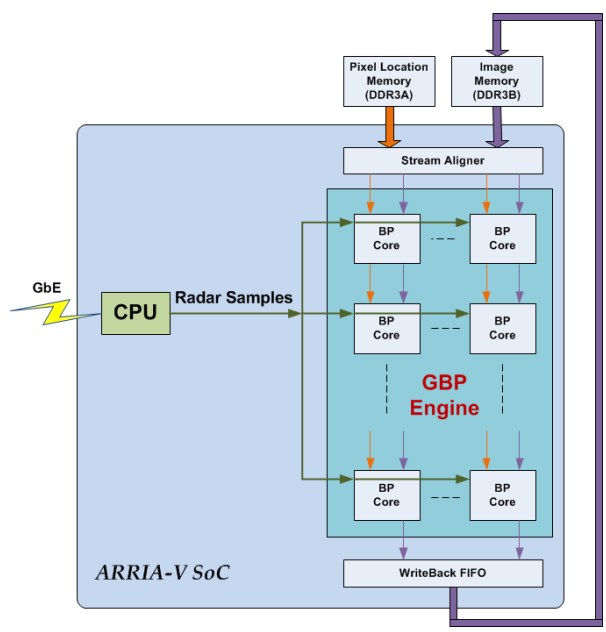

Fig. 2. GBP Engine Block Diagram

The GBP Engine is implemented as a 2D array with a parameterized number of cores in each dimension. Such capability allows optimizing the engine for available memory bandwidth and available computational resources on the FPGA. For example, if the memory interface is able to provide multiple pixels at once, they can be processed simultaneously, along different columns of the array. In contrast, each row of the array represents integration across different aperture points. The cores in the same row calculate the contribution of each aperture point to pixels. Since these cores operate in parallel, they realize parallelization across multiple aperture points. The actual integration is done in a pipelined way. Each core adds its own contribution to a pixel's previous value, and passes it to the next row of the array.

A single FPGA device can't accommodate enough cores to process all aperture points at a single pass. Thus, multiple passes through the array are required. The partially integrated pixels that are pushed out of an array are written back to image memory. Then next iteration is launched to finish the overall integration process. The total number of passes is a ratio between an aperture point number and the y axis dimension of the array. Since the depth of pipeline is negligible in comparison to the number of pixels in the image, the penalty to fill and flush the pipe is very small.

The radar samples are stored inside the core's memory look-up tables (LUT). Different techniques were researched in [5], to optimize the storage. Since the same rows of cores are working on the same aperture point, it's possible to share the memory storage among two columns by utilizing the dual port memories which are available in the FPGA. In this implementation, the LUTs were filled by a processor integrated into the FPGA. The processor obtains the Gotcha dataset from a PC over Ethernet connection and fills them into the core's LUTs. It's interesting to note that because of the flexibility of this FPGA interconnect, the samples can be filled into a whole row of cores, simultaneously. This minimizes overhead associated with the filling process. Radar samples filling mechanism could be also implemented using direct connection of LUTs to radar front-end samplers.

B. Architecture Scalability

Scalability is an important aspect of modern radar systems which need to reuse an architecture across multiple platforms. The described architecture allows scaling the footprint of the image formation to fit the varied requirements. Since the formation is implemented as a parameterized array, it is clear that the number of cores can be reduced or increased, up to the maximum that can fit on a single FPGA device. However, the architecture isn't limited to a single FPGA.

Once the pixels are pushed out of an array, they can be streamed across the next array in a different FPGA device that is attached, as shown in Fig. 3. Integrated High-Speed serial transceivers allow implementing a serial link that chains the devices to create a much bigger virtual systolic array that can spread across multiple FPGA devices. Such chaining enables a scalability to fit very high performance requirements for GBP.

C. Back-Projection (BP) Core

The enabler for the parallel architecture described above is a streaming BP core, shown in Fig. 4. It was implemented with a model-based approach using the DSP Builder Advanced Blockset (DSPBA) tool from Altera. DSPBA allows synthesizing of the design directly from Simulink. Developing and verifying design on a high level allows the performing of different system optimizations related to architecture and numerical accuracy. In addition, the tool would ensure that the synthesized design meets desired performance, by automatically inserting pipelining stages. Fig. 4, show how (1) is mapped into a GBP core architecture.

The Ranging block on Fig. 5 calculates dr.

$$dr = \sqrt{(x - x_k)^2 + (y - y_k)^2 + (z - z_k)^2} - r_c \qquad (2)$$

$dr -$ Differential range from platform to each pixel versus center of swath.

$x_k, y_k, z_k -$ Radar platform location in Cartesian coordinates.

$x, y, z -$ Pixel location in Cartesian coordinates.

$r_c -$ Range to center of the swath from radar platform.

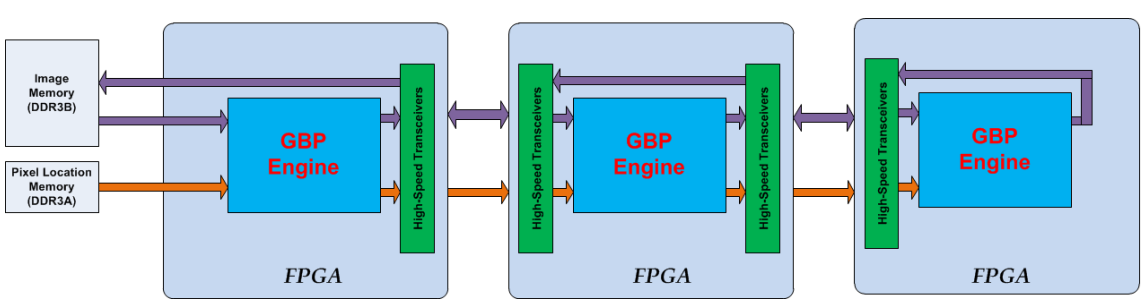

Fig. 3. GBP Architecture Spans across Multiple FPGA Devices

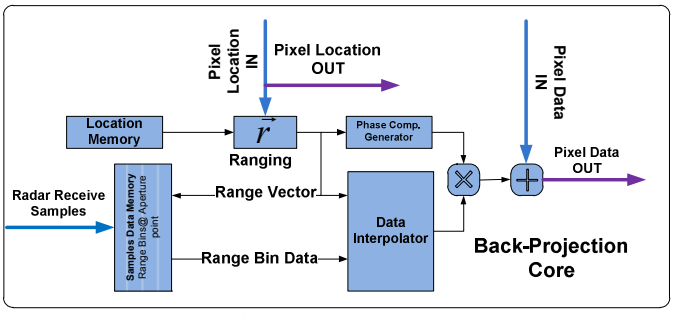

Fig. 4. Back-Projection (BP) Streaming Core

The differential range is used to interpolate the reflected radar echo from the corresponding adjacent range bins. As mentioned above, the linear interpolation is used (3).

$$g(r_k,\theta_k)=g(n)+\frac{g(n+1)-g(n)}{r(n+1)-r(n)}\cdot(r_k-r(n)) \qquad (3)$$

$g(n)-$ Radar sample in adjacent range bin before

$g(n+1)-$ Radar sample in adjacent range bin after

$r(n)-$ Corresponding range to adjacent bin before

$r(n+1)-$ Corresponding range to adjacent bin after

Typically, the range bins are spaced equally, so one can use this assumption to eliminate a division by $r(n+1)-r(n)$ and converting it to multiplication by constant scalar. In addition, the (3) shows how it can be optimized even more if it is converted to range bins scale versus distance. Described concept is shown in (4) and (5).

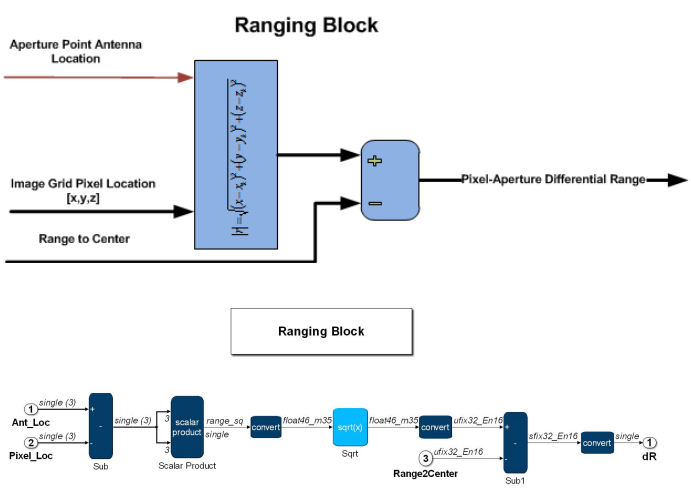

Fig. 5. a) Top: Ranging Block Diagram, b) Bottom: Ranging Block implementation in DSPBA

Such transformation allows reusing calculated indexes as indicated by $r \rightarrow i$ in Fig. 6.

$$dr_RB = dr \cdot \frac{RB_num}{RB_span} \qquad (4)$$

$$g(r_k,\theta_k)=g(n)+(g(n+1)-g(n))\cdot(dr_RB-\lfloor dr_RB \rfloor) \qquad (5)$$

$dr-$ Differential range from platform to each pixel versus center of swath.

dr_RB- Differential range from platform to each pixel versus center of swath scaled to range bins number.

RB_num- Total number of range bins in swath.

RB_span- Span of swath that is covered by RB_num .

It can be seen in (3) that to calculate the interpolated value, we need to obtain two adjacent range bins' complex values from the on-chip memory for each output value. The direct solution can be to store two copies of the radar samples in memory, or to stall the pipe for an additional cycle, in which the second sample is obtained. Neither one of the direct solutions is optimal. In the first, the double memory footprint is consumed, while in the second, the performance is degraded due to a stall in the systolic array. A more optimal solution is shown in Fig. 6. It utilizes the data locality, knowing that the required pair is always around the desired sample. The implementation is done by splitting the memory to two half-size memories, each holding odd and even range bins, respectively. Now that the adjacent samples are stored in different memories, the read bandwidth is effectively doubled. It should be noted that memory address calculation is required along with samples swap capabilities to extract the radar samples in correct order from the memories. Such a solution is using the optimal size of the memory and enables the systolic array operation without stalls.

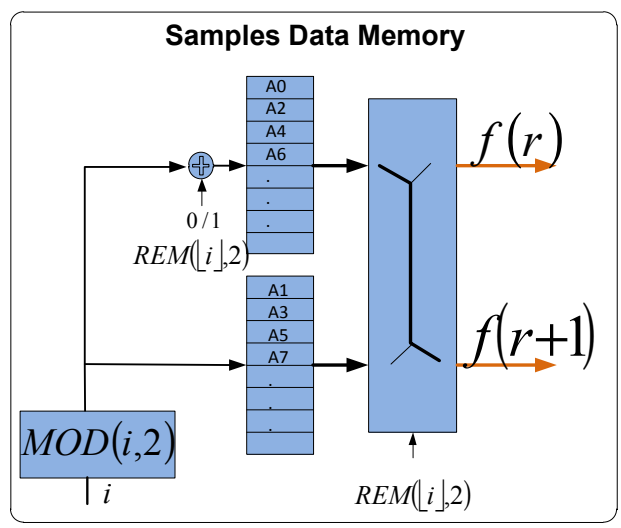

Fig. 6. Radar Samples Data Memory Optimized for Linear Interpolation

Interpolated complex value is multiplied by a phase rolling factor generated using the *SinCos* functional block in DSPBA (6). Then, the result is added to the value of the accumulated complex pixel value from the previous stage of systolic array. It should be noted that along with complex pixel value, the pixel location is pushed to the next stage as well. It eliminates the need to read the value again from the SDRAM memory. Such inherent feature of systolic array architecture on FPGA that allows storage of intermediate results inside pipeline is important advantage over competitive architectures.

$$e^{j\frac{4\pi f_{\min} dr}{c}} = \cos\left(\frac{4\pi}{c}\cdot f_{\min}\cdot dr\right) + j\cdot\sin\left(\frac{4\pi}{c}\cdot f_{\min}\cdot dr\right) \quad (6)$$

$f_{\min}$ - Minimal frequency of radar waveform.

c - Speed of light.

While designing systolic arrays, special attention should be given to the latency that the data incur while passing through cores of the array. This is to ensure different data streams are synchronized while passing through the core. DSPBA automates this process by aligning the streams during design generation. It will automatically insert pipelining stages to compensate for latency incurred by different branches of data flow. Such capability significantly improves the design productivity.

IV. Experimental Results

The described Architecture is based on Arria-V SoC platform from Altera. In addition to the FPGA fabric it integrates an ARM Cortex A9 dual-core processor. The processor was running VxWorks RTOS and to fetch the Dataset from a PC and control the execution of GBP in the GBP Engine. Range bins were pre-computed to form 4,096 range bins per aperture point. The final image size to format was selected to be 512x512 pixels. To achieve cross-range resolution, 1,000 aperture points were used to format each individual image. It was confirmed that the image quality in Fig. 7 is the same as the image quality obtained by high level Matlab implementation of GBP described in [3].

For testing Arria-V SoC development kit from Altera was used. This platform has two DDR3 SDRAM external memory interfaces. In this design the DDR3 interfaces were running at 1,066 Gbps with width of 32 bits each. Internally to the FPGA, the rate was reduced and data parallelized using 256 bit @ 133 MHz. First memory interface generates pixel location stream, while the second streams pixel accumulated complex value. Since second interface should support read and write-back operations, the effective streaming data rate is approximately half of defined memory bandwidth. A single pixel of a complex value using a single IEEE floating point precision requires 64 bits.
Such streaming data rate and data formats imply that the width of systolic array in X dimension should be 2. The logic resources on specified device allowed fitting 20 BP cores into single device.

Thus, the created systolic array could be generated using 10x2 dimensions.

To evaluate the performance of the architecture, the time required to format a single image was measured. To format a single image using the parameters described above took:

$$t_{image} = 120.34ms$$

In order to make a fair comparison to other images, one can introduce performance metric back-projections, or "bp", in an effort required per each single aperture point and for each single pixel. Thus, the system performance is:

$$\frac{1{,}000\cdot 512\cdot 512}{0.12034} = 2.18Gbp/s$$

Besides the absolute performance, the key to efficient implementation resides in power consumption to achieve such performance. In this work, the total power consumption used in execution of the GBP was measured. It includes all the components involved in GBP processing like Arria-V SoC, memory sub-systems, system interfaces, power distribution circuitry etc. This power consumption measured was $P_{total} = 26.55W$. It is desirable to normalize the power by performance to achieve unified scale to compare with other architectures. Here, the unified metric selected is an Energy required in Joules [J].

$$E_{image} = P_{total}\cdot t_{image} = 3.19J$$

$$N_{bp/image} = 1{,}000\cdot 512\cdot 512 = 262.144M$$

$$E_{bp} = \frac{E_{image}}{N_{bp/image}} = 12.17nJ$$

Where $N_{bp/image}$ are a total number of back-projections in the image and E_{bp} is an energy required to calculate a single bp.

The next step of the experiment is to compare achieved metrics using an FPGA to the competitive implementation platforms. For comparison, the Nvidia GeForce GTX 780 GPU card was selected. The back-projection algorithm was implemented and optimized in OpenCL running on the GPU card. Performance was measured in the same way as before, by benchmarking a required time to calculate the single image of similar size, and using same number of aperture points. The power consumption measurement was done differently than before. Since the GPU card resides in the Host server, only the additive power used once the algorithm is executed was measured. The comparison results are presented in Table I.

TABLE I. Competative Benchmarking

Arria-V SoC			GeForce GTX 780		
Performance [Gbp/s]	**Total Power [W]**	E_{bp} [nJ]	**Performance [Gbp/s]**	**Total Power [W]**	E_{bp} [nJ]
2.18	26.55	12.17	4.44	305	68.7

Fig. 7. SAR Image constructed on FPGA using GOTCHA dataset

In addition to described benchmark on Arria-V SoC, this work included projection and benchmark of described architecture on recently released Arria-10 FPGA. This FPGA family realizes the need for high-performance systems that require floating-point calculations. It is extending a functionality of integrated DSP blocks, by adding single precision IEEE complaint floating point operations "hardened" into silicon. As a result there is significant reduction of logic resources required to implement given system with floating-point and therefore ability to pack more processing cores into single device.

To evaluate GBP performance in Arria-10, the BP cores array was implemented. The logic resources required to implement the BP core was declined by 60% versus Arria-V device. Moreover the measured dynamic power efficiency of BP core was improved by 45% versus Arria-V. It should be noted that Arria-10 family has higher density and inherent fabric performance which leads to conclusion that it's achievable GBP performance is 4.1 times higher than the results showed using Arria-V SoC device.

V. CONCLUSIONS

It has been demonstrated that an FPGA is an efficient implementation platform for SAR image formation. Specifically, the research shows that it possible to achieve a scalable, high-performance and power efficient system when implementing a back-projection algorithm. Herein, we show the efficiency comes from using a high-performance fabric, and the features embedded into FPGA silicon including floating-point capabilities in DSP blocks. Power efficiency is a key requirement for modern systems, taking into account the requirements to build SWaP-constrained aerial real-time platforms. The competitive benchmarking shows the energy efficiency ratio of 5.65 and beyond in Arria-V and Arria-10 respectively versus GPGPU mentioned above. The ability to integrate multiple FPGAs into a single card, taking into account the scalability of the architecture, enables building of SWaP-constrained systems that may be impossible to create using other technology.

Future work may evaluate applicability of such a described architecture to Video SAR (ViSAR) applications. ViSAR is a concept of SAR based surveillance that achieves frame update rate > 5 Hz. Such high frame rate allows tracking of fast maneuvering targets using SAR technology and resolution. It is believed that the architecture could achieve smoother frame rate transition, resembling that of a video stream. The implementation could be based on utilizing the previously calculated back-projections and formatting the image in recursive way.

REFERENCES

[1] Hayden, E., Schmalz, M., Chapman, W., Ranka, S., Sahni, S., & Seetharaman, G. (2012, December). Techniques for mapping synthetic aperture radar processing algorithms to multi-GPU clusters. *ISSPIT, 2012*

[2] Cordes, B. Parallel Backprojection: A case study in high performance reconfigurable computing, 2008

[3] Gorham, L. A., & Moore, L. J. SAR image formation toolbox for MATLAB. In *SPIE Defense, Security, and Sensing* , 2010.

[4] Chapman, W., Ranka, S., Sahni, S., Schmalz, M., Majumder, U., Moore, L., B. Parallel processing techniques for the processing of synthetic aperture radar data on GPUs. *ISSPIT, 2011*

[5] Chapman, W., Ranka, S., Sahni, S., Schmalz, M., & Majumder,. Parallel processing techniques for the processing of synthetic aperture radar data on FPGAs. *ISSPIT, 2010*

[6] Benson, T. M., Campbell, D. P., & Cook, D. A.Gigapixel spotlight synthetic aperture radar backprojection using clusters of GPUs and CUDA. *Radar Conference (RADAR), IEEE*, *2012*

[7] Fasih, A., & Hartley, T. GPU-accelerated synthetic aperture radar backprojection in CUDA. *Radar Conference (RADAR), IEEE, 2010*

[8] Gorham, L. A., Majumder, U. K., Buxa, P., Backues, M. J., & Lindgren, A. C. Implementation and analysis of a fast backprojection algorithm. *Defense and Security Symposium*, International Society *for Optics and Photonics, 2006*

[9] Ahmed, I. Study of the local backprojection algorithm for image formation in ultra wideband synthetic aperture radar. *Blekinge Institute of Technology, 2008*

[10] Gorham, L. , Casteel, C. , Gotcha Volumetric SAR Data Set, Version 1.0.